

AI-Mediated Code Comment Improvement

Maria Dhakal · Chia-Yi Su ·
Robert Wallace · Chris Fakhimi ·
Aakash Bansal · Toby Li · Yu Huang ·
Collin McMillan

Received: date / Accepted: date

Abstract This paper describes an approach to improve code comments along different quality axes by rewriting those comments with customized Artificial Intelligence (AI)-based tools. We conduct an empirical study followed by grounded theory qualitative analysis to determine the quality axes to improve. Then we propose a procedure using a Large Language Model (LLM) to rewrite existing code comments along the quality axes. We implement our procedure using GPT-4o, then distil the results into a smaller model capable of being run in-house, so users can maintain data custody. We evaluate both our approach using GPT-4o and the distilled model versions. We show in an evaluation how our procedure improves code comments along the quality axes. We release all data and source code in an online repository for reproducibility.

Keywords source code comments · empirical studies · grounded theory · large language model · code summarization

Maria Dhakal
Department of Computer Science and Engineering, University of Notre Dame, IN, USA.
E-mail: mdhakal@nd.edu

Chia-Yi Su
Department of Computer Science and Engineering, University of Notre Dame, IN, USA.

Robert Wallace
Department of Computer Science and Engineering, University of Notre Dame, IN, USA.

Chris Fakhimi
Department of Computer Science and Engineering, University of Notre Dame, IN, USA.

Aakash Bansal
Division of Computer Science and Engineering, Louisiana State University, LA, USA.

Toby Li
Department of Computer Science and Engineering, University of Notre Dame, IN, USA.

Yu Huang
Department of Computer Science, Vanderbilt University, Nashville, TN, USA

Collin McMillan
Department of Computer Science and Engineering, University of Notre Dame, IN, USA.

1 Introduction

Source code comments are a lynchpin of program comprehension and maintainability (Song et al., 2019). But comments are notorious for being out of date, misleading, and generally of poor quality (Hu et al., 2018). Improving these comments is a valuable target for researchers, with entire research areas having formed variously called code summarization (McBurney and McMillan, 2014; Ahmad et al., 2020), code comment generation (Hu et al., 2018), and niches such as commit message generation (Zhang et al., 2024). Yet a perennial problem in these research areas are the question “what qualities make a code comment good?” and “how do we aim our tools to generate comments along those qualities?” (Song et al., 2019; Rani et al., 2023). Almost all approaches aim to mimic code comments in repositories of human-written source code, but mimicking those comments is likely to produce new comments with the same problems that existed in the comments in the first place.

In this paper, we perform empirical work to understand how comments should improve, then we design and implement a procedure for improving comments. For empirical work, we 1) conduct a study to compare source code comments and obtain programmers’ rationales for why some comments are better than others, and 2) analyze the programmers’ rationales using a grounded theory approach to find key “quality axes” along which code comments should be improved. We then design an approach using a Large Language Model (LLM) to improve existing comments along the quality axes. We implement the approach using both a commercial LLM (GPT-4o (Achiam et al., 2023)) and a smaller model suitable for in-house execution (jam (Su et al., 2023; Su and McMillan, 2024a)). The in-house execution is important because it maintains data custody by not sending source code to a third party.

The niche this paper fills is aligning research on code comment quality with research on code comment generation. Code comment quality has been studied for years; for instance, Khamis et al. (2010) assessed the quality of inline comments based on consistency and language quality using a heuristic based approach. Steidl et al. (2013) evaluated documentation comment quality across four quality attributes, such as consistency, coherence, completeness, and usefulness. Zhou et al. (2017) proposed a heuristic and natural language processing-based technique to detect incomplete and incorrect comments.

Our study is different in that we focus specifically on *improvements* to code comment quality. We ask 10 programmers to read different code comments for the same code and decide on which they prefer. Then we ask them to rewrite the code comment and give a rationale behind their decisions. The result is a dataset of 10 subjective programmer opinion about code quality differences and improvements. Our next step is to find axes of quality along which code comments should be improved. We perform a grounded theory analysis using the Straussian approach (Strauss and Corbin, 1997) to label the rationales given by programmers in our study. We find seven “quality axes” that describe different aspects of improvements that programmers felt were helpful.

We propose an AI-mediated procedure for improving code comments along the seven quality axes we find. Our procedure is essentially to: 1) extract samples of improvements along the quality axes to serve as training examples, 2) fine-tune an LLM-based text generator (e.g., GPT-4o) with the training examples, and 3) distill the output of the LLM into a smaller language model capable of being run in-house (e.g., jam). Then we can use either the fine-tuned LLM or the distilled model to improve arbitrary source code comments along any of the quality axes. Finally, perform an evaluation of our approach on a held out set of source code comments in a study with 70 programmers and show that improvements along the quality axes we propose lead to an overall improvement in perceived code comment quality.

2 Background and Related Work

This section reviews prior work across three key areas relevant to our study: (1) code comment quality, (2) techniques for code summarization and automated comment generation, and (3) the application of grounded theory in software engineering research, with a focus on how qualitative methodologies have been used to derive taxonomies, interpret developer practices, and inform tool design. Table 1 lists recent works related to this paper, highlighting the diversity of papers selected across different domains for our study.

Literature	G	L	C
Wen et al. (2019)	✗	✗	✓
LeClair et al. (2019)	✗	✗	✓
Adolph et al. (2012)	✓	✗	✗
Hoda (2021)	✓	✗	✗
Chen et al. (2021)	✗	✗	✓
Roy et al. (2021)	✓	✗	✗
Rani et al. (2023)	✗	✗	✓
Lee et al. (2023)	✗	✓	✗
Su et al. (2024)	✗	✓	✓
Zhou et al. (2024)	✓	✓	✗
Geng et al. (2024)	✗	✓	✓
Ahmed et al. (2024)	✗	✓	✓
Su and McMillan (2024b)	✗	✓	✗
Su and McMillan (2024a)	✗	✓	✗
This Paper	✓	✓	✓

Table 1: Selection of publications related to this paper from the last five years. All approaches cited above are based on code comment improvement using neural networks and the use of grounded theory in one way or another.

Column *G* means the use of the grounded theory approach. *L* means LLM-based code summarization and comment generation. *C* means code comments generation and quality assessment.

2.1 Studies of Code Comment Quality

Code comment quality is essential for ensuring the readability and maintainability of source code. Prior work has examined various aspects of assessing and evaluating comment quality, such as their adequacy (Arthur and Stevens, 1989), content quality (Khamis et al., 2010; Steidl et al., 2013), the co-evolution of comments and code (Fluri et al., 2009), and the detection of inconsistent comments (Ratol and Robillard, 2017; Wen et al., 2019). Several studies have proposed tools and techniques for the automatic assessment of comment quality (Khamis et al., 2010; Steidl et al., 2013; Sun et al., 2016; Zhou et al., 2017). In a recent systematic literature review, Rani et al. (2023) cataloged 21 quality attributes (QAs), yet what constitutes a “good” comment remains a topic of debate. Our work addresses this gap by identifying the quality dimensions using the grounded theory approach, listed in Table 2, and improving the comments along those quality dimensions.

2.2 Code Summarization & Comment Generation

Source code summarization is the task of generating concise natural language descriptions of source code (Haiduc et al., 2010). It is a subset of code comment generation, where comments may describe larger codebases more broadly, while a “summary” specifically denotes a well-defined, high-level description of a particular code section (e.g., a method or class) (Su et al., 2024).

Code Summarization: Source code summarization has been an active area of research in software engineering for over a decade (Su et al., 2024; Haiduc et al., 2010). Its goal is to abstract the functionality of code into a human-readable format, helping developers understand the purpose and behavior of code, particularly within large codebases (Su et al., 2024). Early efforts employed traditional rule-based approaches (Sridhara et al., 2011; McBurney and McMillan, 2014; Rodeghero et al., 2014). More recently, neural network-based models (Bansal et al., 2023; LeClair et al., 2019, 2020) and large language models (LLMs) such as OpenAI’s GPT-4 (Achiam et al., 2023), CodeLlama (Roziere et al., 2023), and CodeBERT (Feng et al., 2020) have significantly advanced this task (Su et al., 2024). Despite these advancements, large models often fail to generate summaries that fully address categories such as “what,” “why,” “how-to-use,” “how-it-is-done,” “property,” and “others” Chen et al. (2021); Sun et al. (2024). To address this limitation, we propose a reinforcement learning from human feedback (RLHF) Lee et al. (2023) method to generate code comments aligned with the quality axes defined in Table 2 by finetuning a smaller model, which is suitable for in-house deployment.

Comment Generation: Complementing code summarization, automated comment generation focuses on improving or revising existing comments. Code comments are widely recognized as valuable aids for program comprehension (Sridhara et al., 2011; Hu et al., 2018). Early studies primarily used template-based (Sridhara et al., 2010; McBurney and McMillan, 2014) and information retrieval (IR)-based methods (Haiduc et al., 2010; Wong et al., 2013) for comment generation. Since 2017, neural models (Liu et al., 2018; Hu et al., 2018;

LeClair et al., 2019; Wang et al., 2020; Wei et al., 2020; Mu et al., 2022, 2023) and large language models has advanced in the task to generate comments automatically. Recent work shows that large language models (LLMs) can leverage in-context learning to produce multi-intent comments (Geng et al., 2024; Sun et al., 2024; Haider et al., 2024). However, these models primarily aim to imitate human-written comments and often lack a nuanced understanding of comment quality (Rani et al., 2023; Geng et al., 2024; Mu et al., 2023). This misalignment motivates our work, which integrates comment quality considerations into the improvement process. Our goal is to bridge the gap between comment quality research and generation techniques, enabling method to generate code comments aligned with the quality axes defined in Table 2.

2.3 Grounded Theory in Software Engineering

Grounded Theory (GT) is a methodology for inductively generating theory from data (Glaser and Strauss, 1967). Unlike approaches that test existing theories, GT focuses on theory generation (Adolph et al., 2012), as introduced by Glaser and Strauss (1967) in *The Discovery of Grounded Theory*. In software engineering, GT is commonly used to analyze qualitative data—such as developer interviews and surveys—to explain phenomena like collaboration or code evaluation (Coleman and O’Connor, 2007; Carver, 2004; Stol et al., 2016). It is particularly effective for exploratory questions like “What’s going on here?” (Adolph et al., 2012), allowing patterns and concepts to emerge from observations (Stol et al., 2016). Given the lack of established theory on code comment quality (Rani et al., 2023), we adopt GT to derive a set of “quality axes” based on developer feedback and rationale. Among the three GT variants discussed by Stol et al. (2016), we use the Straussian variant, which supports structured coding and allows guiding frameworks (Glaser and Strauss, 1967). Straussian GT involves three coding stages (Stol et al., 2016; Strauss and Corbin, 1997):

- **Open Coding** is the process of generating “categories” and examining their dimensional variations (Stol et al., 2016). As outlined in Sec. 4.1, we adopted an iterative approach in which human annotators independently proposed labels, followed by review sessions to merge and refine them into a finalized set.
- **Axial Coding** involves reassembling the data after open coding by identifying relationships among the categories (Glaser, 1992). As described in Sec. 4.2, we conducted a series of review sessions to group the labels into broader themes.
- **Selective/Subjective Coding** is the process of determining a central category that integrates all major categories (Corbin and Strauss, 1998). Following the method proposed by Strauss and Corbin, we identified an umbrella label that encapsulates the key factors underlying the rationales.

3 Comment Quality Rationale Survey

This section describes our survey to collect programmers’ rationales about code comment quality differences. In a nutshell, we show programmers Java methods and comments that describe those methods. Then we ask programmers to rank the comments by their quality, then rewrite the “better” comment to improve it further. We ask the programmers to write a rationale for their decisions about comment quality. The purpose is to coax programmers to think critically about code comment quality and provide their subjective opinions.

3.1 Research Method

Our research method is a web-based survey with several pages. The survey shows a series of Java methods. There are two pages in the survey per method. On the first page shown in Figure 1, the human study participant sees a Java method’s source code and two comments describing the method. The participant then selects the comment that he or she prefers. An option is provided for no preference, but this option is discouraged in the instructions.

Function getString:

```
public String getString(String bundleName, String key, String defaultValue) {
    ResourceBundle bundle = getResourceBundle(bundleName);
    String value = null;
    try {
        if (bundle != null) value = bundle.getString(key);
    } catch (Exception e) {}
    return (value != null) ? value : defaultValue;
}
```

Comment 1:
retrieves a string value from a resource bundle based on a given key, and returns a default value if the key is not found or an exception occurs

Comment 2:
retrieves a string value associated with a given bundle name, key, and default value, and returns it if it exists, otherwise it returns the default value

Overall, which summary is better in your opinion?
☐ Comment1 ☐ Comment2 ☐ Cannot Decide

Submit

Fig. 1: First page of the study interface.

Function getString:

```
public String getString(String bundleName, String key, String defaultValue) {
    ResourceBundle bundle = getResourceBundle(bundleName);
    String value = null;
    try {
        if (bundle != null) value = bundle.getString(key);
    } catch (Exception e) {}
    return (value != null) ? value : defaultValue;
}
```

Comment 1:
Enter your comment here...

Rationale:
Explain your rationale...

Submit

Fig. 2: Second page of the study interface.

After the participant indicates a preference (or none) for a comment, a second page shown in Figure 2 asks the participant to rewrite the comment that the participant chose (the survey chooses randomly if no preference is offered) to improve the comment further. The participant is also asked to provide a brief rationale about the participant’s decision about quality. After finishing the task and clicking submit, the participant is shown another Java method. The survey continues for a total of 35 Java methods. We chose 35 methods because we found it takes approximately 90 minutes for participants to complete, after which various studies report a risk of fatigue bias (Bansal et al., 2021; McBurney et al., 2016; Sridhara et al., 2010).

3.2 Source Code & Comments Dataset

The origin of the Java methods and comments in our study is the dataset released by (Su et al., 2023) for source code summarization, which in turn is based on a dataset recommended by (LeClair et al., 2019) and refined by (Bansal et al., 2021). The refinements addressed issues in datasets of code summaries raised by (Shi et al., 2022) and (Allamanis, 2019). We chose this dataset due to its extensive vetting and history in related research, as well as its large size and scope. While it is tempting to view the dataset as “one programming language” (Java), it includes projects that represent a wide range of domains, programmers, and level of maintenance.

The dataset already included code summaries written by people for each Java method, but (Su and McMillan, 2024a) added comments generated by GPT-3.5 and a smaller language model that distilled GPT-3.5, for a total of three choices of comment for each method. The dataset also includes a subset of around 8k Java methods that are even more carefully curated for quality and recommended as a test set in code summarization experiments. We configured our survey to randomly select Java methods from the this 8k test set (again, up to 35 total). Then, we randomly pick two of the three available summaries to display to the study participant for evaluation.

Note that our goal is to collect programmers’ rationales for code comment quality. Given the subjective nature of these opinions, we do not consider it appropriate to enforce agreement. Reasonable persons may disagree about preferences – what we care about and analyze is the rationale.

3.3 Participants

We recruited 80 programmers from the Prolific platform (Prolific, 2025) who were at least 25 years of age, were located in the United States or United Kingdom (for payment purposes), who had a university degree in Computer Science or Computer Engineering, and who self-report experience in Java. Ten programmers were assigned to each of the eight task: one comment quality rationale survey, and seven to evaluate the best comment along the axis. Remuneration was between \$20 and \$60/hour, depending on local market rates.

Table 2: Labels from our Grounded Theory Process

#	Description	Label / Code	Axial Code	Subjective Code
1	The preferred comment has additional information about the code logic of the method. The rationale may say things like: “more detailed about how the method works” or “more detailed about how the algorithm is implemented.”	logical	internalizing	
2	The preferred comment has a specific variable name or data type, i.e., a very small additional that makes the comment less vague. The rationale may point out that the comment refers specifically to “VariableName” which makes it more detailed or clear.	precise	internalizing	
3	The preferred comment provides additional information that clarifies a feature of the method. For example, in a graphical program, “fills a rectangular area” versus “fills a rectangular area <i>with a specific color</i> .”	unambiguous	internalizing	
4	The preferred comment provides qualitatively more information in general without specifying information categorized in another label. The rationale may just say “provides all required details” or “much more detailed overall.”	exhaustive	internalizing	refocusing

5	The preferred comment is considered better because it clarifies an error or exception type, reasons why an error or exception might occur, or examples of inputs that could cause a failure.	troubleshooting	externalizing	
6	The preferred comment includes information to help understand why the method exists or what it is doing. For example, a method may play sounds, but a comment may contextualize the method by explaining that the purpose of the sound is for a game.	contextualizing	externalizing	
7	The preferred comment is more concise than the non-preferred comment. Note that at times a comment may be more concise if it includes more focused information, even if the comment is longer in number of words.	condensing	externalizing	

8	Refers to rationales that are too vague to understand or contain non sequiturs, including potentially fraudulent survey responses. We remove these from further analysis.	toss	n/a	n/a

3.4 Threats to Validity

This study is subject to validity threats common to survey-based research in software engineering. Differences in the source code and comments, participant selection, or the survey interface could influence the results. To mitigate these risks, we sample from a large, vetted dataset and use a simple web interface to minimize complexity (unlike an IDE). While web surveys pose fraud risks (Danilova et al., 2021), traditional countermeasures like challenge questions are increasingly ineffective due to tools like ChatGPT and Copilot (Ghorbani et al., 2023). Instead, we manually inspect and remove potentially fraudulent responses in the next section.

4 Analysis of Comment Quality Rationale

This section describes our qualitative analysis of the rationales we collected in the study in the previous section. We use a Straussian grounded theory strategy described by Strauss and Corbin (1997) and further elucidated for software engineering research in several articles (Adolph et al., 2012; Coleman and O'Connor, 2007; Hoda, 2021; Díaz et al., 2023) and especially by Stol et al. (2016). The Straussian grounded theory process is divided into three main steps: open coding, axial coding, and selective coding. Grounded theory approach has been effectively applied in software engineering to support collaborative qualitative analysis, for instance, Díaz et al. (2023)'s work on inter-rater reliability and agreement in collaborative grounded theory studies. While a complete review of grounded theory is beyond the scope of this paper, the following subsections summarize the key steps of the approach and describe our specific implementation.

4.1 Open Coding

Open coding is an approach in which annotators read subject artifacts and label each artifact with a single word or phrase. The labels are defined by the annotators and are not from a predefined set. The annotators may work independently, but then meet in review sessions to combine the labels. The goal is to continue independent work followed by group meetings until a final set of labels is decided. In our analysis, the subject artifacts are the rationales given in the survey in the previous study, and the annotators are the authors of this paper. We pose the following research question and started our analysis individually and then in group review sessions:

RQ1 What are the factors that influence programmer opinions about code comment quality?

Table 2 shows the outcome of our open coding process as a set of labels we attach to each rationale. After finalizing the set of labels, we re-annotated each rationale with this final set. In principle, a rationale could have more than one label, or a comment could be improved in multiple ways. However, in our review sessions, we applied a dominant label to each rationale and resolved disagreements through discussion. This discussion was necessary because the highly subjective nature of rationales means multiple interpretations could be correct; the group review phase is intended to form a collective opinion about the most pertinent information. Authors such as Delgado and Tibau (2019) and Donker et al. (1993) argue against calculating Cohen's Kappa and other agreement metrics in similar scenarios, relying instead on collaborative discussions to resolve disagreement.

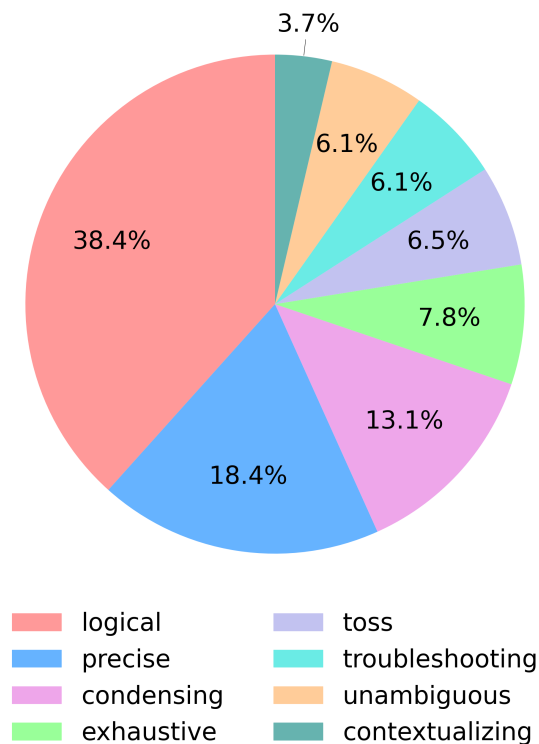


Fig. 3: Distribution of tags from our annotation process.

4.2 Axial Coding

The axial coding phase of grounded theory involves organizing the labels generated during open coding into higher-level categories that capture the relationships among the open codes. We carried out this process through a series of group review meetings. As a result, we identified two key themes as axial codes: *internalizing* and *externalizing*. The internalizing category encompasses labels referring to specific details within the method being described, whereas the externalizing category includes labels referring to information beyond or primarily affecting the method. For example, an internalizing improvement to a code description might address aspects such as variable names or the algorithms implemented within a method, such as the type of search algorithm employed. In contrast, an externalizing improvement might incorporate information explaining why the search is necessary in the program or identifying inputs from other components that could cause the search to fail. Note that we assigned the “condensing” label to the externalizing axial code group, as it consistently involved removing unnecessary details about the internal workings of the method in favor of emphasizing higher-level contextual information.

4.3 Subjective Coding and Theory Summary

The purpose of the subjective coding phase is to place the axial and open codes under a single umbrella code (the subjective code) that describes the key purpose behind the subject artifacts. This subjective code serves as the lynchpin of the theory developed in the grounded theory process. We found a subjective code of “refocusing” that crystallizes the key factors behind the rationales from our survey. We define refocusing as changing the information in the source code comment to better align with one of several qualities that programmers need from the comments. At a very high level, code comments improve when they discuss information along one of the qualities that we identify as labels during the open coding part of the grounded theory process.

The application of the labels is subjective and can vary depending on programmer preferences and the source code being described by the comment. Consider the pie chart in Figure 3. Approximately 38% of the labels are “logical” which on hand is not surprising because many comments that programmers write are focused on how an algorithm works or other implementation details. But on the other hand, more than 60% of the tags focus on other quality criteria, showing that programmers look for more than implementation details in comments. The second-most common tag was “precise”, with around 18% of improvements related to very specific details such as the inclusion of a particular method name. Note however that these tags are indicators of why study participants felt one summary was better than another – they should not be interpreted as indicating what programmers need in an ideal case. For example, the low percent for “contextualizing” (around 4%) most likely indicates that contextual information is rare in summaries, rather than suggesting that programmers do not need contextual information.

4.4 Examples

To clarify the distinctions among labels, we present the illustrative examples. Table 3 demonstrates how annotators assign the labels for rationales and Table 4 illustrates the code, and summary along the quality axes.

Table 3 illustrates the rationale labeling scheme. Rationale statements that address the behavior or functionality of the code are labeled as logical, while statements that emphasizes critical implementation details, such as excluding itsPreviewLocalizatorLayer when removing layers, and specifying that the upper activity layer is added only when an active project exists, making the comment more accurate and informative is labeled as precise.

To further illustrate this categorization, Table 4 presents a Java code snippet along with corresponding summaries aligned with different quality axes. Each summary highlights a distinct aspect of the same code. The precise summary identifies the Ymsg object and emphasizes clarity in explanation, the logical summary describes the core functionality, the contextualize summary provides additional context—such as “writing the appropriate byte array to the connection”, and the unambiguous summary is clear and free of ambiguity.

Table 3: Examples of rationale and label

<u>Label</u>	<u>Rationale</u>
logical	Providing more clarity and specificity of the behavior of the code.
contextualizing	provide a more complete and specific description of the function’s behavior
unambiguous	Providing more clarity about the storage manager and the nature of the loading process
precise	Mention the exclusion of the itsPreviewLocalizatorLayer when removing layers from the camera. Additionally, I emphasize that the upper activity layer is added to the camera conditionally, specifically when an active project exists

Table 4: Examples of label and summary

```

public void rejectContact(String buddy, String message) throws
IOException {
    YmsgPacket yp = new YmsgPacket(version, 0, Event.
REJECTCONTACT, 0);
    yp.addData(new YmsgData(1, username));
    yp.addData(new YmsgData(7, buddy));
    yp.addData(new YmsgData(14, message));
    connection.write(yp.toByteArray());
}

```

<u>Label</u>	<u>Summary</u>
contextualize	Rejects a contact by transforming the chat message into an YmsgPacket, writing the Ymsg object, and ultimately writing the appropriate byte array to the connection.
logical	Rejects a contact by creating a YmsgPacket with specified parameters and writing it to the connection.
unambiguous	rejects a contact by creating a YmsgPacket with specified parameters, adding data to it, and then writing it to the connection
precise	Rejects a contact by transforming the chat message into an YmsgPacket, writing the Ymsg object, and handling the response.

5 Improvement Approach

Our approach defines a systematic process for fine-tuning language models to generate code summaries optimized for one of the seven quality axes identified in the previous section. An overview of this process is presented in Figure 4. The procedure involves three main steps: (1) using a pre-trained base model to generate multiple candidate summaries, (2) selecting the candidate that best aligns with a specified quality axis, and (3) fine-tuning the base model based on these selections. The result is a refined model capable of producing code summaries tailored to a specific quality dimension.

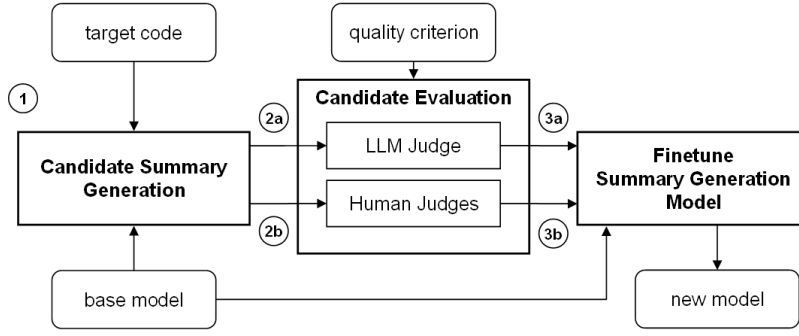


Fig. 4: An overview of the three steps in our approach and their inputs.

5.1 Candidate Summary Generation

The first step in our approach is to generate n candidate summaries for each section of target code from a code dataset (Figure 4, area 1). The purpose of this dataset is to train/fine-tune a model and is not the code for which summaries will be generated by a user of the new model. In our implementation, we used the Java dataset from Su et al. (2023) that we also used in Section 3.2, though in principle many datasets are possible. We randomly selected 45k methods from training samples in that dataset. Then we used the base model to generate $n = 4$ candidate summaries for each of the 45k methods.

5.2 Evaluation of Candidate Summaries

The second step is to pick one of the n candidate summaries that is best along a given quality axis. We had two parts to this selection. First (Figure 4, area 2a) we use a commercial LLM (GPT-4o in our implementation) to improve the summary in a “reinforcement learning from AI feedback” (RLAIF) strategy (Lee et al., 2023; Madaan et al., 2024) which has been suggested in certain software engineering contexts (Ahmed et al., 2024). We create a prompt based on the description of each quality criterion from the previous section. Each prompt has $n = 4$ candidate summaries and the code for a Java method, as the example prompt shows in Figure 5 (see our online appendix for all prompts, Section 8). The prompt asks the LLM to choose the best of the four options along a particular quality axis. We use this prompt to select the best candidate for 44.5k of the 45k Java methods. The result is one summary for each quality axis for each of the 44.5k methods.

The second part of the evaluation is to ask human experts to select the best summary along each quality axis (Figure 4, area 2b) in a “reinforcement learning from human feedback” (RLHF) strategy (Lee et al., 2023). We perform a survey for each quality axis. The survey is similar to the one we used in Section 3.1, except that it asks participants to choose a summary out of $n = 4$ candidates and it provides instructions around each quality criterion (rather than overall personal preference). We recruited ten participants for

each quality criterion survey in the same manner as in Section 3.3, but we restricted recruitment such that each person could only participate in one survey to reduce potential anchoring bias (Furnham and Boo, 2011). Each survey showed 50 Java methods with summary candidates for a total of 500 samples. These methods were randomly selected from the pool of 45,000 methods for which candidate summaries were initially generated. Note that none of the 500 methods used in the surveys overlapped with the 44,500 methods used during the automated evaluation phase with the LLM.

Basically our idea is to rely on the AI feedback for quantity and then human feedback for quality. One can affordably obtain several thousand responses from the commercial LLM, but human subjective judgment is expensive and not practical at large scale. Note that we are certainly not the first to use RLAIIF or RLHF for model improvement, but instead demonstrate one way to combine them for code summarization.

5.3 Finetuning the Summary Generation Model

Our third step is to create a summary generation model for each quality criterion. Our process is to start with an existing summary generation model as a base model, then fine-tune that model using the selections of candidates from the previous section. We did finetuning using the 44.5k samples selected by the LLM judge first (Figure 4, area 3a), then finetuned again using 400 of the 500 samples selected by the human judges (the other 100 we held out as a test set for the evaluation in the next section). Our rationale for using the LLM selections first was to give the model a baseline with sufficient data sizes to generate code summaries along a particular quality criterion, even if the LLM selections are not “perfect.” However, we also used selections from human judges last, to modify the model’s output with the best examples at the end of the process.

Table 5: Fine-tuning hyperparameters in our experiments.

Parameter	JAM	CodeLlama
Epochs	3	3
Learning Rate	3.0×10^{-5}	1.0×10^{-4}
Batch Size	4	2
Gradient Accumulation Steps	32	16
LoRA r	—	64
LoRA α	—	16
LoRA Dropout	—	0
Quantization Type	—	nf4
Bits	—	4

You are an expert Java developer with a focus on clear, concise, and accurate documentation. Your task is to evaluate multiple summaries of a given Java method and select the one that best captures the method's core functionality, input specifications, primary operations, and error handling. When evaluating each summary, consider the following criteria:

1. **Clarity and Precision:** The ideal summary should be easy to understand and should precisely describe what the method does. It should avoid vague language and accurately represent the method's purpose.
2. **Appropriate Use of Variable Names and Data Types :** Specific variable names or data types should be included only when they enhance understanding or remove ambiguity. Unnecessary use of variable names or data types should be avoided to keep the summary focused and clear.
3. **Coverage of Key Components :**
 - **Functionality :** The summary should capture the main purpose of the method without extraneous details.
 - **Input Specifications :** Mention any key inputs, including data types and any constraints if needed.
 - **Primary Operations :** Describe the main operations performed within the method, such as loops, conditional checks, or transformations, if they relate to the purpose and help to clarify functionality of code.
 - **Error Management :** If applicable, include any error handling or edge case management in a concise way.
4. **Avoid Redundancy :** Try to exclude any descriptions or variable references that don't enhance the reader's understanding if possible. The summary should be streamlined to include essential information.

Here is the Java code and the summaries:

Java Code: {code}

Summaries:

1. {summaries[0]}
2. {summaries[1]}
3. {summaries[2]}
4. {summaries[3]}
5. Absolutely cannot decide.

Please select the most precise summary according to the criteria above:

- Use specific variable names or data types only when they clarify the functionality or remove ambiguity.
- Choose the clearest and most informative summary that best defines the overall functionality of the code.

If you cannot confidently choose the best summary, select option 5 ('Absolutely cannot decide') as a last resort. Return only the content of the summary you select, without adding extra information, numbers, or notes.

Fig. 5: Prompt template we used for the first-pass LLM-based improvement for the “precise” quality axis. We replace {summaries[x]} and {code} tags with a summary candidate ($0 \leq x \leq n$, where n is the number of candidates) and the method's Java source code.

5.4 Implementation Notes

We built two implementations to demonstrate our approach. The implementations are identical except for the base model. One implementation uses `jam` model released by Su and McMillan (2024a), and the second uses the 13B parameter version of `codellama` model by Meta (Roziere et al., 2023). We used the `jam` model because 1) it is small enough to be finetuned without LoRA (Hu et al., 2022) or other “parameter efficient” procedures on a single GPU, which preserves data custody and reduces experimental variables, 2) was shown to still be large enough to produce code summaries on par with a commercial LLM (Su and McMillan, 2024a; Bansal et al., 2021), and 3) is open-source with pretraining data published so we can control experimental variables. We used the `codellama` model because 1) it represents a much larger model than `jam` (37x larger) designed for code intelligence tasks, 2) is still feasible to finetune on commercial hardware with LoRA (Hu et al., 2022) procedures, and 3) is partially open-source.

Note we used the finetuning settings recommended by the creators of each model with the hyperparameters in Table 5. For `jam`, Su and McMillan (2024a) published recommendations specifically for code summarization tasks. For `codellama`, we used default settings the creators recommended for several tasks because no specifics for code summarization were provided.

Our implementation platform comprised an Intel i9-10900X CPU, 256GB of system memory, and four NVidia A5000 GPUs, each with 24GB of video memory. The software environment included Python 3.10.12, PyTorch 2.0.1, TensorFlow 2.12.0, and CUDA 12.2.

6 Evaluation

Our research objective in this evaluation is to determine if our approach from the previous section leads to models that generate summaries better adapted to each quality criterion. By “better adapted” we mean summaries that are more like those that people selected as higher quality along a quality axis than summaries generated prior to finetuning with our approach. Therefore we ask the research question (RQ):

RQ2 Are summaries from models finetuned with our approach more like summaries selected for quality axes, than base models prior to finetuning, according to accepted code summary similarity metrics?

The rationale behind RQ2 is that the finetuning process should lead the model to generate summaries more like the ones the model saw during finetuning, and to evaluate the process we need to measure the similarity of the summaries from the finetuned and base models to a reference set. The reference is the set of 100 selections by human participants in the survey in the previous section (recall there were 500 selections made by human participants, of which 400 were used for finetuning and 100 set aside for evaluation). For this evaluation, we use the 100 set-aside Java methods as input to both finetuned and

base models. For each of those 100 methods we have samples selected as better along each quality axis. We use text similarity metrics to compare the output of each model to these samples. Because quality criterion differences may be subjective and involve nuances of language, we follow the suggestion of Haque et al. (2022) and use semantic similarity measures (as opposed to n-gram comparison metrics). Specifically, we calculate BERTScore (we report F1) (Zhang et al., 2019) and USE (Haque et al., 2022). Following Roy et al. (2021) we also do Mann-Whitney tests to determine the statistical significance between scores for each model. The Mann-Whitney test is appropriate because it is a paired, non-parametric test. A paired test is appropriate because each model uses the same set of input Java methods. A non-parametric test is appropriate because it is conservative with respect to distribution assumptions.

Threats to validity exist in this evaluation, though these are consistent with similar studies. Key threats to validity include: 1) the set of Java methods, 2) the human participants who made the selections, and 3) the similarity metrics. In theory, differences in these categories could lead to differences in the evaluation outcomes. We mitigate the threat from the Java methods by choosing these methods from a large dataset that has been vetted in numerous previous papers (Su et al., 2023). We mitigate the threat from the human participants by having a total of 50 participants to reduce the impact of bias from any one person, and implementing quality control procedures described in Section 3.1. We mitigate the threat from metrics by using two well-studied metrics that have been specifically recommended for code summary evaluation (Zhang et al., 2019; Haque et al., 2022), and by using recommended statistical tests (Mann and Whitney, 1947). A limitation to our study, though not necessarily a threat to validity, is that the reference set is selected as “best” along a particular category, though may or may not be the best in a theoretical sense. In theory, a better summary could be written along a particular quality axis than the one chosen as best of n . This is a limitation of all RLHF and RLAI approaches and not unique to our study.

7 Evaluation Results

Tables 6 and 7 show our evaluation results we use to answer RQ2. Overall we observe strong improvement in terms of similarity to human-selected improved summaries as measured by automated metrics. For **jam** we observe statistically-significant improvement in BERTScore for five of the seven quality axes and four of the seven for USE. The mean level of improvement in BERTScore is 23% and in USE is 3.6%. For **code llama** we observe statistically-significant improvement in both BERTScore and USE for all axes. The mean level of improvement in BERTScore is 115% and in USE is 30%. These results indicate higher similarity to human-selected improved summaries.

A few observations stand out among the results, however. First, for **jam**, the axis with the worst performance was “condensing” for which we found no statistically-significant difference for either metric (BERTScore even reported

Table 6: Metrics and statistical test results for RQ2 for jam.

(a) BERTScore (F1)				
	Baseline	Finetuned	p-value	% diff
unambiguous	45.95	52.2	0.0155	13.60
condensing	47.34	47.02	0.64	-0.68
contextualizing	44.76	49.69	0.067	11.01
exhaustive	47.15	52.07	0.032	10.43
logical	42.93	49.66	0.0036	15.68
precise	47.64	53.7	0.0044	12.72
troubleshooting	45.41	91.51	<0.0001	101.52

(b) USE				
	Baseline	Finetuned	p-value	% diff
unambiguous	71.5175	76.0375	0.0187	6.32
condensing	71.3375	72.2375	0.48	1.26
contextualizing	70.74	75.75	0.01	7.08
exhaustive	71.0975	74.26	0.08	4.45
logical	69.4775	72.915	0.06	4.95
precise	71.85	75.7175	0.035	5.38
troubleshooting	69.72	71.8125	<0.0001	3.00

Table 7: Metrics and statistical test results for RQ2 for CodeLLAMA

(a) BERTScore (F1)				
	Baseline	Finetuned	p-value	% diff
Unambiguous	26.82	53.61	<0.0001	99.89
Condensing	22.63	51.16	<0.0001	126.07
Contextualizing	21.60	49.39	<0.0001	128.66
Exhaustive	25.89	39.31	<0.0001	51.83
Logical	21.36	47.43	<0.0001	122.05
Precise	22.93	54.66	<0.0001	138.38
Troubleshooting	21.35	50.42	<0.0001	136.12

(b) USE				
	Baseline	Finetuned	p-value	% diff
Unambiguous	59.05	76.28	<0.0001	29.18
Condensing	56.32	72.24	<0.0001	28.27
Contextualizing	56.85	72.37	<0.0001	27.30
Exhaustive	57.57	66.91	<0.0001	16.22
Logical	55.24	71.92	<0.0001	30.20
Precise	54.31	76.54	<0.0001	40.93
Troubleshooting	53.93	74.81	<0.0001	38.72

a slightly negative difference). We attribute this finding to the human-selected summaries shorter length for condensing: while people tended to select shorter summaries in terms of number of words for this quality axis, the finetuning process tended to yield summaries with different content and word usage than summaries with fewer (or more) words, on average. A second observation is that the improvement levels for `jam` are much smaller than for `codellama`. We attribute this finding to higher diversity in the initial summaries from `codellama`, meaning that the human-selected summaries are more different from the other initial summaries than in `jam`. The higher diversity is likely

due to the larger model size and larger dataset used to pretrain `codellama`. The `jam` model builders focused exclusively on a curated dataset of Java, while `codellama` was trained using many languages and repositories. One may view this as `codellama` having more “headroom” to improve both in terms of difference from initial summaries and in capacity to learn due to model size.

A third observation is that the improvement along “troubleshooting” for `jam` as measured by BERTScore is much higher than for USE or that we found for `codellama`. We attribute this finding to `jam`’s tendency to copy the exception type name directly from the code into the summary, which was also prominent in the human-selected summaries for that quality axis. BERTScore calculates similarity by aggregating semantic similarity of each token, so identical word usage leads to high scores. In contrast, USE calculates semantic similarity of an entire sentence, so the contribution of a single identical word match is less important to the total score. Therefore, when `jam` learned during finetuning to copy a particular token from the source code, it closely matches what humans selected in terms of word usage, which was detected more strongly in BERTScore than in USE.

Table 8: Examples from `jam` model after finetuning

```
protected void notifyProcessingReasonlet(ReasonletInstance
instance) {
    if(listeners == null) {
        return;
    }
    for(ReasonletContainerListener listener : listeners) {
        try {
            listener.processingReasonlet(instance);
        } catch(Exception e) {
            if( logger.isEnabledFor(Level.ERROR) ) {
                logger.error( String.format("error while notifying
listener %s for processingReasonlet", listener), e);
            }
        }
    }
}
```

Reference Summary: notifies all registered instances of processing reasonlet and handles any exceptions thrown during the process

Label	Improved Summary
Condensing	notifies all registered ReasonletContainerListeners of a ProcessingReasonlet instance
Contextualize	notifies all listeners of processing a Reasonlet instance by invoking the processingReasonlet method on each listener
Logical	notifies all listeners of processing a Reasonlet instance by invoking their processingReasonlet method

Table 9: Examples from codellama model after finetuning

```

public Intervention peek(int i) {
    if (i < 1)
        throw new IndexOutOfBoundsException("Index " + i + "
        is lower than 1");

    if (i > this.interventionList.size())
        throw new IndexOutOfBoundsException("Index " + i + "
        is greater than stack size: " + this.interventionList.size());

    return interventionList.getFirst();
}

```

Reference Summary: returns the first element in a stack of interprets, but throws an exception if the provided index is out of bounds

Label	Improved Summary
Logical	returns the first element of a list of intervention objects, throwing an exception if the index is out of bounds
Precise	returns the first element of a list of intervention objects at a specified index, throwing an exception if the index is out of bounds
Exhaustive	returns the first element of a linked list, throwing an exception if the given index is less than 1 or greater than or equal to the size of the list
Troubleshoot	returns the first element of a stack of intervention objects, after checking if the index is within the bounds of the stack

Tables 8 and 9 present illustrative examples, each comprising a Java code snippet, a reference summary selected by a human evaluator from a set of candidate summaries, and the corresponding improved summaries produced after fine-tuning. These examples demonstrate how the fine-tuned models enhance the clarity, specificity, and overall quality of code comments across various evaluation axes. Table 8 focuses on clarifying vague or ambiguous descriptions. For instance, the reference summary—"notifies all registered instances of processing reasonlet"—is vague and lacks sufficient context. The refined summaries address these limitations by being more concise and informative. Specifically, the "Condensing" axis promotes brevity while preserving meaning; "Contextualize" introduces the specific listener type and the method being invoked; and "Logical" ensures the comment clearly articulates the intended action and its target. Table 9 further illustrates improvements across multiple quality dimensions. The reference summary in this example vaguely refers to a "stack of interprets," which is both unclear and likely inaccurate. In contrast, the refined summaries improve terminology and detail: the "Logical" and "Precise" axes enhance accuracy and specificity; "Exhaustive" introduces explicit exception-handling conditions; and "Troubleshoot" emphasizes boundary checks, resulting in more robust and informative comments.

8 Conclusion

This paper advances the state-of-the-art in two key ways:

1. We conduct a grounded theory analysis of differences in code summaries to determine seven “quality axes” along which people view improvements in code summaries.
2. We present and evaluate a strategy for fine-tuning language models to automatically improve code summaries along these seven quality axes.

The intellectual merit/novelty of this paper is in the empirical findings from our grounded theory analysis, in the combination of RLAIIF and RLHF techniques for code summary improvements, and in demonstrating that these techniques result in summaries that improve along the quality axes we found during the grounded theory analysis. We measured improvement in terms of similarity to human-selected summaries using two metrics (one focusing on word usage similarity and one focusing on overall sentence similarity) and applied to two language models. Overall we found stronger improvement in `codellama` as opposed to `jam`, which may be expected considering `codellama`’s larger size and pretraining dataset.

Declarations

Ethical approval

This study was conducted in accordance with institutional guidelines and approved by the Institutional Review Board.

Informed consent

We presented the informed consent on the first page of the survey website, and all participants agreed to the study.

Author Contributions

All authors contributed to the study’s conception and design. Material preparation, data collection, and analysis were performed by Maria Dhakal, Chia-Yi Su, and Aakash Bansal. Maria Dhakal and Collin McMillan drafted the manuscript, and all authors reviewed and approved the final version.

Code / Data Availability Statement

We release all data, scripts, survey materials, and results at https://github.com/apc1-research/improving_summary

Conflict of Interest

The authors declare no conflict of interest.

Clinical trial number: not applicable.

References

- Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, et al. (2023) Gpt-4 technical report. arXiv preprint arXiv:230308774
- Adolph S, Kruchten P, Hall W (2012) Reconciling perspectives: A grounded theory of how people manage the process of software development. *Journal of Systems and Software* 85(6):1269–1286, DOI <https://doi.org/10.1016/j.jss.2012.01.059>, URL <https://www.sciencedirect.com/science/article/pii/S0164121212000386>, special Issue: Agile Development
- Ahmad WU, Chakraborty S, Ray B, Chang KW (2020) A transformer-based approach for source code summarization. arXiv preprint arXiv:200500653
- Ahmed T, Devanbu P, Treude C, Pradel M (2024) Can llms replace manual annotation of software engineering artifacts? arXiv preprint arXiv:240805534
- Allamanis M (2019) The adverse effects of code duplication in machine learning models of code. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pp 143–153
- Arthur JD, Stevens KT (1989) Assessing the adequacy of documentation through document quality indicators. In: *Proceedings. Conference on Software Maintenance-1989*, IEEE, pp 40–49
- Bansal A, Haque S, McMillan C (2021) Project-level encoding for neural source code summarization of subroutines. In: *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, IEEE, pp 253–264
- Bansal A, Jiang S, Haque S, McMillan C (2023) Statement-based memory for neural source code summarization. arXiv preprint arXiv:230711709
- Carver JC (2004) The impact of background and experience on software inspections. *Empirical Software Engineering* 9:259–262, URL <https://api.semanticscholar.org/CorpusID:7616856>
- Chen Q, Xia X, Hu H, Lo D, Li S (2021) Why my code summarization model does not work: Code comment improvement with category prediction. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30(2):1–29
- Coleman G, O'Connor R (2007) Using grounded theory to understand software process improvement: A study of irish software product companies. *Information and Software Technology* 49(6):654–667
- Corbin J, Strauss A (1998) *Basics of qualitative research: Techniques and procedures for developing grounded theory*. Sage, 2nd Ed
- Danilova A, Naiakshina A, Horstmann S, Smith M (2021) Do you really code? designing and evaluating screening questions for online surveys with programmers. In: *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, IEEE, pp 537–548
- Delgado R, Tibau XA (2019) Why cohen's kappa should be avoided as performance measure in classification. *PloS one* 14(9):e0222916
- Díaz J, Pérez J, Gallardo C, González-Prieto Á (2023) Applying inter-rater reliability and agreement in collaborative grounded theory studies in software

- engineering. *Journal of Systems and Software* 195:111520
- Donker D, Hasman A, Van Geijn H (1993) Interpretation of low kappa values. *International journal of bio-medical computing* 33(1):55–64
- Feng Z, Guo D, Tang D, Duan N, Feng X, Gong M, Shou L, Qin B, Liu T, Jiang D, et al. (2020) Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:200208155*
- Fluri B, Würsch M, Giger E, Gall HC (2009) Analyzing the co-evolution of comments and source code. *Software Quality Journal* 17:367–394
- Furnham A, Boo HC (2011) A literature review of the anchoring effect. *The journal of socio-economics* 40(1):35–42
- Geng M, Wang S, Dong D, Wang H, Li G, Jin Z, Mao X, Liao X (2024) Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pp 1–13
- Ghorbani A, Cassee N, Robinson D, Alami A, Ernst NA, Serebrenik A, Wasowski A (2023) Autonomy is an acquired taste: Exploring developer preferences for github bots. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE, pp 1405–1417
- Glaser B (1992) *Basics of Grounded Theory Analysis*. Sociology Press
- Glaser B, Strauss A (1967) *The discovery of grounded theory: Strategies for qualitative research*. New York: Aldine De Gruyter.
- Haider MA, Mostofa AB, Mosaddek SSB, Iqbal A, Ahmed T (2024) Prompting and fine-tuning large language models for automated code review comment generation. *arXiv preprint arXiv:241110129*
- Haiduc S, Aponte J, Marcus A (2010) Supporting program comprehension with source code summarization. In: *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 2*, pp 223–226
- Haque S, Eberhart Z, Bansal A, McMillan C (2022) Semantic similarity metrics for evaluating source code summarization. In: *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, pp 36–47
- Hoda R (2021) Socio-technical grounded theory for software engineering. *IEEE Transactions on Software Engineering* 48(10):3808–3832
- Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W, et al. (2022) Lora: Low-rank adaptation of large language models. *ICLR* 1(2):3
- Hu X, Li G, Xia X, Lo D, Jin Z (2018) Deep code comment generation. In: *Proceedings of the 26th conference on program comprehension*, pp 200–210
- Khamis N, Witte R, Rilling J (2010) Automatic quality assessment of source code comments: the javadocminer. In: *Natural Language Processing and Information Systems: 15th International Conference on Applications of Natural Language to Information Systems, NLDB 2010, Cardiff, UK, June 23-25, 2010. Proceedings 15*, Springer, pp 68–79
- LeClair A, Jiang S, McMillan C (2019) A neural model for generating natural language summaries of program subroutines. In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, IEEE, pp 795–806

- LeClair A, Haque S, Wu L, McMillan C (2020) Improved code summarization via a graph neural network. In: Proceedings of the 28th international conference on program comprehension, pp 184–195
- Lee H, Phatale S, Mansoor H, Lu KR, Mesnard T, Ferret J, Bishop C, Hall E, Carbune V, Rastogi A (2023) Rlaif: Scaling reinforcement learning from human feedback with ai feedback, arXiv e-prints
- Liu Z, Xia X, Hassan AE, Lo D, Xing Z, Wang X (2018) Neural-machine-translation-based commit message generation: how far are we? In: Proceedings of the 33rd ACM/IEEE international conference on automated software engineering, pp 373–384
- Madaan A, Tandon N, Gupta P, Hallinan S, Gao L, Wiegrefe S, Alon U, Dziri N, Prabhume S, Yang Y, et al. (2024) Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36
- Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* pp 50–60
- McBurney PW, McMillan C (2014) Automatic documentation generation via source code summarization of method context. In: Proceedings of the 22nd International Conference on Program Comprehension, pp 279–290
- McBurney PW, Liu C, McMillan C (2016) Automated feature discovery via sentence selection and source code summarization. *Journal of Software: Evolution and Process* 28(2):120–145
- Mu F, Chen X, Shi L, Wang S, Wang Q (2022) Automatic comment generation via multi-pass deliberation. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp 1–12
- Mu F, Chen X, Shi L, Wang S, Wang Q (2023) Developer-intent driven code comment generation. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp 768–780
- Prolific (2025) Prolific: An online participant recruitment platform. <https://www.prolific.com/>
- Rani P, Blasi A, Stulova N, Panichella S, Gorla A, Nierstrasz O (2023) A decade of code comment quality assessment: A systematic literature review. *Journal of Systems and Software* 195:111515
- Ratol IK, Robillard MP (2017) Detecting fragile comments. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 112–122
- Rodeghero P, McMillan C, McBurney PW, Bosch N, D’Mello S (2014) Improving automated source code summarization via an eye-tracking study of programmers. In: Proceedings of the 36th international conference on Software engineering, pp 390–401
- Roy D, Fakhoury S, Arnaoudova V (2021) Reassessing automatic evaluation metrics for code summarization tasks. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp 1105–1116
- Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Sauvestre R, Remez T, et al. (2023) Code llama: Open foundation models

- for code. arXiv preprint arXiv:230812950
- Shi L, Mu F, Chen X, Wang S, Wang J, Yang Y, Li G, Xia X, Wang Q (2022) Are we building on the rock? on the importance of data preprocessing for code summarization. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp 107–119
- Song X, Sun H, Wang X, Yan J (2019) A survey of automatic generation of source code comments: Algorithms and techniques. *IEEE access* 7:111411–111428
- Sridhara G, Hill E, Muppaneni D, Pollock L, Vijay-Shanker K (2010) Towards automatically generating summary comments for java methods. In: Proceedings of the 25th IEEE/ACM international conference on Automated software engineering, pp 43–52
- Sridhara G, Pollock L, Vijay-Shanker K (2011) Automatically detecting and describing high level actions within methods. In: Proceedings of the 33rd International Conference on Software Engineering, pp 101–110
- Steidl D, Hummel B, Juergens E (2013) Quality analysis of source code comments. In: 2013 21st international conference on program comprehension (icpc), Ieee, pp 83–92
- Stol KJ, Ralph P, Fitzgerald B (2016) Grounded theory in software engineering research: a critical review and guidelines. In: Proceedings of the 38th International conference on software engineering, pp 120–131
- Strauss A, Corbin JM (1997) Grounded theory in practice. Sage
- Su CY, McMillan C (2024a) Distilled gpt for source code summarization. *Automated Software Engineering* 31(1):22
- Su CY, McMillan C (2024b) Semantic similarity loss for neural source code summarization. *Journal of Software: Evolution and Process* 36(11):e2706
- Su CY, Bansal A, Jain V, Ghanavati S, McMillan C (2023) A language model of java methods with train/test deduplication. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp 2152–2156
- Su CY, Bansal A, Huang Y, Li TJJ, McMillan C (2024) Context-aware code summary generation. arXiv preprint arXiv:240809006
- Sun W, Miao Y, Li Y, Zhang H, Fang C, Liu Y, Deng G, Liu Y, Chen Z (2024) Source code summarization in the era of large language models. arXiv preprint arXiv:240707959
- Sun X, Geng Q, Lo D, Duan Y, Liu X, Li B (2016) Code comment quality analysis and improvement recommendation: An automated approach. *International Journal of Software Engineering and Knowledge Engineering* 26(06):981–1000
- Wang W, Zhang Y, Sui Y, Wan Y, Zhao Z, Wu J, Yu PS, Xu G (2020) Reinforcement-learning-guided source code summarization using hierarchical attention. *IEEE Transactions on software Engineering* 48(1):102–119
- Wei B, Li Y, Li G, Xia X, Jin Z (2020) Retrieve and refine: exemplar-based neural comment generation. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, pp 349–360

- Wen F, Nagy C, Bavota G, Lanza M (2019) A large-scale empirical study on code-comment inconsistencies. In: 2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC), IEEE, pp 53–64
- Wong E, Yang J, Tan L (2013) Autocomment: Mining question and answer sites for automatic comment generation. In: 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 562–567
- Zhang T, Kishore V, Wu F, Weinberger KQ, Artzi Y (2019) Bertscore: Evaluating text generation with bert. arXiv preprint arXiv:190409675
- Zhang Y, Qiu Z, Stol KJ, Zhu W, Zhu J, Tian Y, Liu H (2024) Automatic commit message generation: A critical review and directions for future work. *IEEE Transactions on Software Engineering* 50(4):816–835
- Zhou Y, Gu R, Chen T, Huang Z, Panichella S, Gall H (2017) Analyzing apis documentation and code to detect directive defects. In: 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), IEEE, pp 27–37
- Zhou Y, Yuan Y, Huang K, Hu X (2024) Can chatgpt perform a grounded theory approach to do risk analysis? an empirical study. *Journal of Management Information Systems* 41(4):982–1015